Multimedia Tools and Applications (2025) 84:34179–34205
https://doi.org/10.1007/s11042-025-20612-9

Procedural game level generation with GANs: potential, weaknesses, and unresolved challenges in the literature

Daniele F. Silva[1,2] · **Rafael P. Torchelsen**[2] · **Marilton S. Aguiar**[2]

Received: 12 June 2024 / Revised: 13 December 2024 / Accepted: 8 January 2025 /
Published online: 18 January 2025

Abstract
Procedural content generation (PCG) has significantly impacted game design by automating the creation of dynamic game environments, thereby saving time and effort while maintaining the freshness at each play which is required for games as a service. Recent advances in machine learning, particularly Generative Adversarial Networks (GANs), offer exciting possibilities for generating diverse and playable game levels that surpass traditional methods. Despite challenges such as training instability and ensuring playability, GANs present considerable potential for dynamic content generation. This paper explores the advantages of GAN-based approaches, addresses their limitations, and suggests improvement strategies, including combining algorithms or using solvers to mitigate poor generations. This paper explores the advantages and limitations of GAN-based approaches and suggests promising research directions to improve GAN-based procedural level generation, including combining algorithms or using solvers to mitigate poor generations. Future research directions are also identified, such as the need for user studies and improved GAN training techniques to fully harness the potential of GANs in game-level generation.

Keywords Procedural content generation · Generative adversarial network · Game level · Level generation

1 Introduction

Procedural Content Generation (PCG) is employed in game design for the generation of game content, such as levels, terrains, characters, and items [1], enabling designers to create dynamic and heterogeneous gaming environments. While manual content creation requires

✉ Daniele F. Silva
daniele.fernandes@iffarroupilha.edu.br

Rafael P. Torchelsen
rafael.torchelsen@inf.ufpel.edu.br

Marilton S. Aguiar
marilton@inf.ufpel.edu.br

1 Federal Institute of Education, Science and Technology Farroupilha, Alegrete, Rio Grande do Sul, Brazil

2 Federal University of Pelotas, Pelotas, Rio Grande do Sul, Brazil

substantial time and effort from designers to develop static levels, PCG automates this process, assisting the rapid generation of content dynamically, even for large-scale games, with minimal human intervention.

These algorithms can be customized to accommodate various game genres and styles, ranging from platformers to role-playing games. Several popular games in recent years use PCG as the backbone of their design, as these games are intended to be replayed multiple times, allowing for ongoing monetization. Examples include Helldivers 2[1], Starfield[2], and No Man's Sky[3].

Over time, these algorithms have harnessed several techniques to generate levels, including grammar-based approaches [2–4], evolutionary algorithms [5–8], cellular automata [9, 10], and constraint-based methods [11–13]. Grammar-based approaches employ predefined rules and structures to generate levels, supporting the creation of intricate and organized environments. Evolutionary algorithms traverse the space of possible level configurations, optimizing for specific criteria such as difficulty or playability. Cellular automata algorithms represent levels as grids of cells, evolving them over time to create dynamic and aesthetically pleasing landscapes. Constraint-based methods impose specific constraints or objectives during the level-generation process, ensuring that the generated levels meet the desired criteria.

Algorithms that generate levels with the optimal balance of challenge [14], variety, and engagement can be intricate and time-consuming. However, recent advancements in machine learning, particularly Generative Adversarial Networks (GANs), present novel possibilities for game-level generation. Generative methods, such as Variational Autoencoders (VAEs), Diffusion models, and GANs, have increasingly been applied in various domains to generate high-quality data. Among these, GANs have gained significant attention due to their ability to approximate the distribution of data presented to the model. GAN models can learn the complex representations of patterns and constraints inherent in handcrafted level designs, creating new levels that are diverse and playable without the need for explicitly encoded game design rules.

Despite the promising capabilities of GANs in image processing [15–18], their adoption in the games industry remains limited due to unresolved challenges and limitations associated with level generation. These challenges include the instability of training procedures [19, 20], the potential for mode collapse [21], and the necessity to adjust constraints to ensure levels are not unbeatable.

Compared to other techniques in Procedural Content Generation via Machine Learning applied to level design, it is still early to determine which approach is superior, due to the limited number of studies directly comparing these different methods [22]. Nevertheless, GANs continue to exhibit significant strengths, including their application in hybrid approaches within game design and other domains [23].

Our survey focuses specifically on GANs to assist researchers interested in the game level generation domain. Through a comprehensive analysis of GAN-based approaches in existing literature, we explore the potential of employing GANs for procedural-level generation in games. Moreover, we discuss the advantages of GAN-based approaches over traditional PCG methods, including their capability to capture intricate level features and generate more realistic and varied environments. Additionally, we address the limitations of employing GANs for level generation and demonstrate how GANs can surmount some of these challenges.

[1] https://www.playstation.com/games/helldivers-2/

[2] https://bethesda.net/game/starfield

[3] https://www.nomanssky.com/

The structure of this survey is as follows: Section 2 provides an overview of state-of-the-art surveys in PCG. Section 3 outlines the fundamentals of GANs, their architectural and training advancements, and discusses their strengths, challenges, and limitations. In Section 4, we categorize the primary games identified in the literature and discuss the mechanics and specific challenges of each genre within the context of the GAN generation. Section 5 discusses GAN implementation issues, such as data processing and training, and how state-of-the-art methods address these problems. Finally, Section 6 concludes the work by summarizing key findings and suggesting directions for future research.

2 State-of-the-art overview of procedural content generation

In the literature, several surveys elucidate advancements in PCG within gaming and offer valuable insights into specific game content generation. Hendrikx et al. [24] introduce a six-layered taxonomy of game content and map PCG methods to these game content layers. For newcomers to the field, this is an excellent starting point for understanding various PCG approaches within the gaming domain. Similarly, Smelik et al.'s [25] survey grouped works by virtual worlds content, such as: terrains, vegetation, rivers, roads, buildings and cities. Shaker et al. [26] present the first textbook on PCG in games, delivering a comprehensive overview of the research field.

Additional surveys have delved into the primary works within the field. Summerville et al. [27] examine machine learning models trained on existing content to generate game content, emphasizing applications such as autonomous generation and co-creativity. The authors name this class of PCG algorithms as Procedural Content Generation via Machine Learning (PGCML), a topic that was explored by Guzdial et al. [13] in their book, where they examine various PCGML approaches for generating video game content. Furthermore, Liu et al. [28] survey various deep learning methods applied to game content generation and discuss potential future directions, including mixed-initiative generation and style transfer.

Some surveys have focused on specific game genres, such as dungeon games [29, 30] or puzzle games [31]. Other surveys have concentrated on specific types of algorithms, such as evolutionary and metaheuristic search algorithms [5] and knowledge transformation [32]. We recognize that numerous techniques, such as VAEs and Reinforcement Learning (RL) [33–35], are still being employed to generate levels using machine learning. However, our review focuses on GANs and how adversarial training demonstrates the ability to produce valid and diverse samples.

Despite the mentioned surveys, we have found no surveys discussing GAN-PCG approaches specifically applied to game level design, which we consider important to address. The only exception is the Hughes et al.' s work [36], who briefly mention a few works on level generation using GANs, but do not explore the specific aspects involved in level generation.

3 Generative adversarial networks

A GAN is an artificial intelligence framework introduced by Ian Goodfellow in 2014 [37]. It comprises two neural networks, the *generator* and the *discriminator*, which are trained simultaneously through adversarial training. The generator aims to create synthetic data for a given distribution, while the discriminator seeks to differentiate between real and synthetic

data. During training, the generator improves at producing data that resembles the target distribution, striving to make its outputs (synthesized data) indistinguishable from real data by the discriminator.

The literature has introduced numerous architectural variations and loss functions to enhance GAN performance and achieve better convergence. For instance, replacing traditional fully connected networks (VanillaGAN) with deep convolutional networks (DCGAN) has emerged as a promising strategy, as demonstrated by Radford et al. [38]. Furthermore, adopting loss functions based on the Wasserstein divergence measure (WGAN) has significantly improved model adaptation and performance, as Gulrajani et al. [39].

However, the primary challenge with GANs is maintaining stability and convergence during training. Regularization techniques have been proposed to bolster training stability to address this challenge. These techniques include the integration of batch normalization or dropout layers [38], gradient penalty [39] in WGAN-GP, and spectral normalization [40] in SN-DCGAN. These methods aim to alleviate common problems in GAN training [19, 20].

3.1 Advantages, challenges, and limitations

GANs can produce innovative and creative outputs by learning from data distributions, making them valuable for tasks such as content generation. Additionally, GAN training is unsupervised, enabling the models to learn from unlabeled data. This feature broadens their applicability, making GANs useful across various domains, including level generation in the digital game industry.

However, GANs face significant challenges during training and implementation. The instability problem, for instance, makes GANs susceptible to mode collapse or oscillations in the training process. Mode collapse occurs when the generator fails to capture the entire data distribution, resulting in limited or repetitive outputs. Additionally, evaluating the quality of generated samples is challenging, as traditional metrics may not fully capture a specific domain's fidelity, diversity, or quality. Furthermore, GANs are highly sensitive to hyperparameters, necessitating careful tuning and numerous training iterations to achieve the desired results.

Some limitations of GANs include accurately capturing complex data distributions, particularly in imbalanced or rare data [41, 42]. Through research in the field of image processing, it has been established that the performance of GANs heavily depends on the quality and diversity of training data, limiting its effectiveness in domains with limited or biased datasets [43]. Evaluating GAN performance presents another hurdle, as conventional model evaluation techniques may not fully capture its complexities and nuances [44].

Despite the inherent challenges associated with GANs, their usage persists in the literature due to the alternative methods limitations such as VAEs and RL. VAEs, while offering enhanced robustness and control over latent space representations, tend to generate outputs with lower fidelity compared to GANs. Moreover, RL approaches, particularly in Procedural Content Generation via Reinforcement Learning (PCGRL), demonstrate superior adaptability and control over generation objectives. However, these methods often require extensive computational resources and prolonged training periods. To address these limitations, some studies have explored hybrid approaches that combine the strengths of these methods while mitigating their weaknesses. By integrating GANs with VAEs [34, 45, 46] or incorporating RL strategies [47].

4 Applications of GANs in games

In the literature on PCG using GANs, we find a variety of works focused on level generation for different commercial games, including DOOM [48, 49], Super Mario Bros. [50–55], The Legend of Zelda [53, 56], Mega Man [57], and Minecraft [58]). There are also applications in independent (indie) games, which include those developed by smaller studios or created for academic purposes (e.g., ENGAGE [59], Keiki [60]) and similar works like terrain generation [61, 62] or structure generation [63, 64] applied to games. These instances underscore the efficacy of GANs and their potential for scalability in the game design process. Such works explore different game genres, including platform, dungeon, puzzle, educational and shoot 'em up games.

Although all referenced works appear in every table, their discussion varies according to their primary contributions: Section 4 addresses works with greater applied impact, while Section 5 focuses on those with a stronger emphasis on GAN-related issues.

The most discussed game genres in PCG-GANs are platform, dungeon, and puzzle games. Each genre has distinct gameplay mechanics, level structures, and challenges that can be generated using GANs. We delve into an in-depth discussion about the specific challenges associated with each genre.

4.1 Platform games

This genre comprises levels with platforms suspended in the air or other environments. Generally, the player completes the level by reaching the endpoint while avoiding obstacles and enemies. Player movement can be side or vertical scrolling, where the character moves horizontally and vertically.

Side-scrolling is defined based on the player's movements from left to right on the screen while the environment scrolls, revealing new elements of the level. In vertical scrolling, the player moves vertically through the screen, typically progressing upwards. Mechanics such as climbing, jumping, or flying enable the player to advance. Popular side-scrolling games include Super Mario Bros. and Mega Man, while Lode Runner exemplifies a vertical-scrolling game.

The generation of levels for platform games still presents several open problems. A key challenge for GANs is maintaining the cohesion of game elements, ensuring that generated structures are logically organized. For example, in Super Mario Bros., faulty pipes should not disrupt gameplay (Fig. 1), and in Lode Runner, stairs should be reachable for players to climb.

Steckel et al. [65] address the distribution of elements problem using evolutionary algorithms to explore the latent space trained with GANs. Meanwhile, Di Liello et al. [66] propose a GAN-based architecture with constraints to better guide learning structural and reachable elements, producing playable levels.

Another issue arises with non-linear levels, which do not follow a straightforward linearity of player movement. It is challenging to control the flow of connectivity between adjacent level segments (Fig. 2). Although this problem is more evident in non-linear levels, it is also present in side- or vertical-scrolling games generated by segments.

To tackle the connectivity problem, works typically devise methods for preprocessing the data input to train a GAN effectively. Capps et al. [57] specialized different GAN models to learn the representation of each segment based on its direction. Wang et al. [67] use the latent

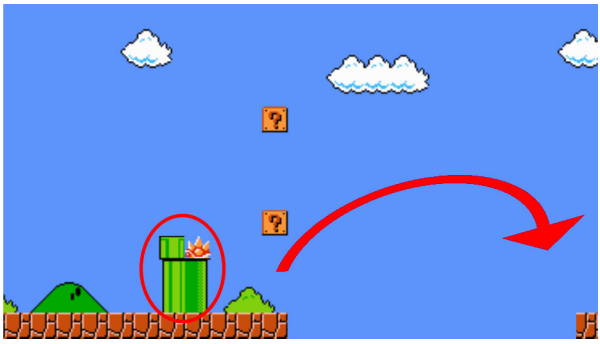

Fig. 1 It is a challenge for GANs to generate elements that cohesively define the level structure. On the left side, faulty pipes are generated; on the right side, unreachable platforms are generated with a normal jump

space of a GAN-trained model and a reinforcement learning algorithm to generate levels that combine level features with music features. Connectivity and difficulty adjustment are performed using past segments and player experiences to generate a new segment. Nam et al.

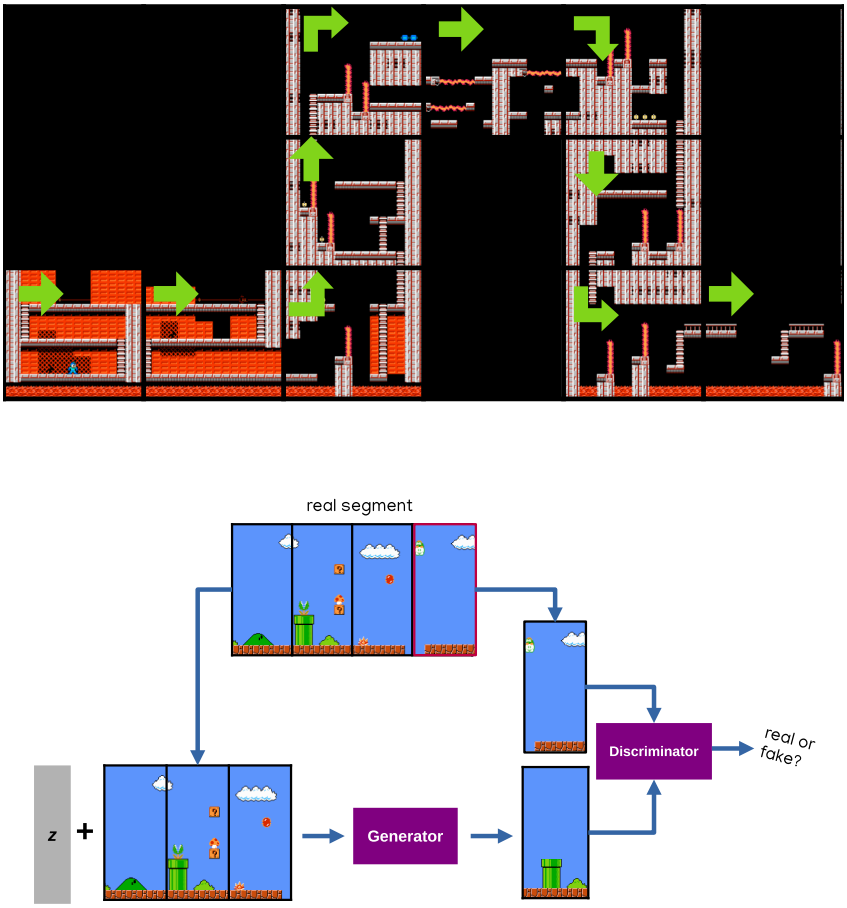

Fig. 2 Difficulty arises in connecting different map segments, either by following the segment orientation (top) [57] or using a part of the previous segment to generate the next one (bottom) [52]

[52] approach generation continuously, training the GAN so that the input is a level segment and the model returns this segment filled with a new fragment.

Generating content with GANs can be challenging due to limited control over the output. It is difficult to tailor the results to specific design requirements, such as adjusting elements in levels and modifying difficulty [47]. Some studies employ evolutionary algorithms to explore the latent space of a GAN model, supporting the adjustment of generated levels. Exploration through latent space is crucial for refining results and can be conducted either manually [68] or automatically [53, 69]. Interaction between the human and machine designers for content generation is called a mixed-initiative approach [70].

4.2 Dungeon games

The dungeon game genre typically involves exploring labyrinthine environments, often called dungeons or caves, filled with traps, puzzles, enemies, and collectibles. Due to the intricate layout characteristics, with the possibility of multiple reachable paths and interconnected rooms, various methods have been proposed in the literature to ensure connectivity with adjacent segments or paths for complete levels.

Adjacent segments or interconnected rooms (Fig. 3) refer to configurations similar to those in The Legend of Zelda. Like platform games, connectivity is also a significant gameplay challenge for dungeon games. Gutierrez et al. [56] combine GANs with the Graph Grammar algorithm to assist in connectivity and the arrangement of elements within levels. Unreachable paths (Fig. 4) are problems commonly observed when generating complete levels, as seen in works involving the game Doom [48, 49, 71], the indie game in the work of [72, 73], and the educational game ENGAGE [59]. The terrain layout is important, and the arrangement of game elements is also essential for the level composition. Level repairs are intended to ensure playability if any rules are not met during the generation of the level or segment/room [56, 74].

Another solution, distinct from the previously presented, involves learning the game's dynamics. In the work of Kim et al. [76], they propose a generative model for simulating player dynamics by generating the next frame of the game. However, the model can produce segments with invalid structures and is still limited to the original Pac-Man game mechanics, excluding enemies from the scenario. Despite these limitations, it represents an important step in learning gameplay dynamics.

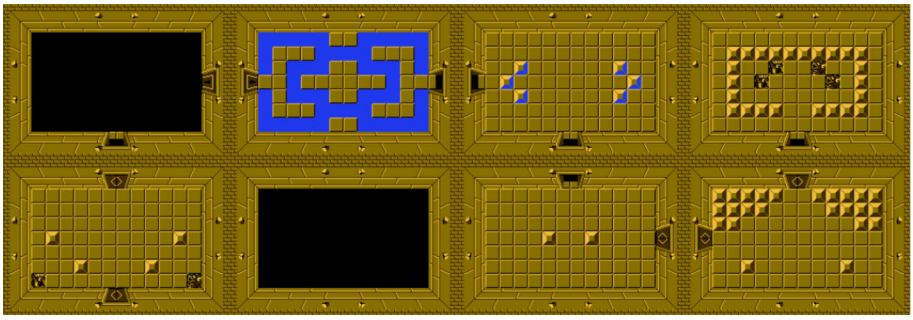

Fig. 3 The set of rooms represents a single level. Rooms from Video Game Level Corpus [75]

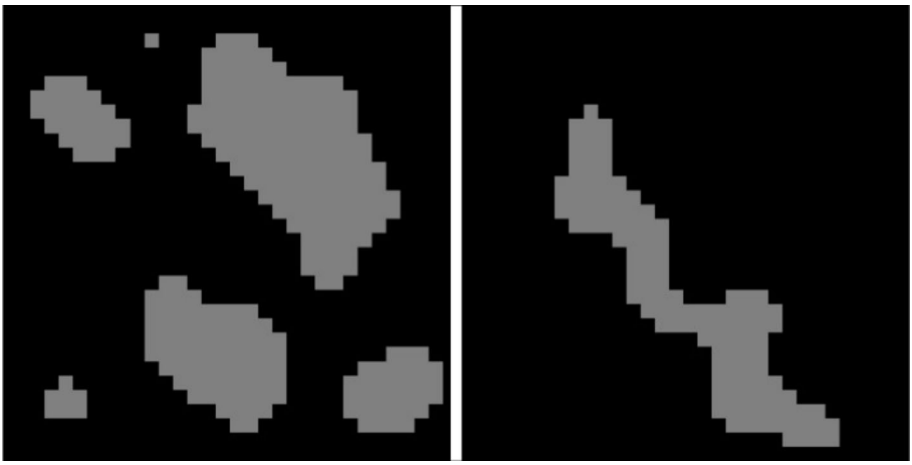

Fig. 4 Terrain generation of an indie game [73]. On the left side, the generated terrain contains many unreachable paths (gray), while on the right side, the terrain has a possible path between two points inside the gray region

4.3 Puzzle games

This game genre challenges players' problem-solving skills and logic through puzzles or mental challenges. These games often require players to use critical thinking, pattern recognition, and creativity to progress through levels or scenarios. There is a diverse range of puzzle-solving experiences, which includes traditional physics-based challenges [63] and block-matching tasks [77, 78].

Hald et al. [77] present a mixed-initiative approach to designing levels based on the specifications of game designers. This approach includes a shape sketch to design the level layout and a piece distribution vector to control the generation. In the work discussed in the previous section, Puzzle elements can also be incorporated into other genre games, as dungeon. In this context, Gutierrez et al. [56] use Graph Grammar for connectivity and for organizing each room's content. This graph structure can enable the construction of puzzles that the player needs to solve.

An important aspect of level generation is evaluating the quality of generated samples, with various studies addressing this validation differently depending on the game's specificity. For the Angry Birds game, Abraham et al. [63] apply heuristics to validate the number of blocks for structural composition. However, assessing stability is not straightforward, as it requires game initialization to calculate block movements over time.

4.4 Other genres

The racing game genre revolves around competitive racing among players. These games typically focus on speed, skillful maneuvering, and sometimes vehicle customization. In this genre, levels are composed of different track layouts, terrain features, and obstacles, providing a diverse experience for the player. Awiszus et al. [54] train a model to generate tracks for Super Mario Kart, but this is not a simple task. If the connectivity rule is not applied to the track, it may lead to results with dead-end paths.

In works exploring terrain generation [62], the focus often lies on creating maps that closely resemble the topological organization of real geographical spaces. Open-world games as the Grand Theft Auto series frequently make use of real-based maps. According to Nunes et al. [46], building more realistic maps, such as cities, is time-consuming. Although PCG techniques help save time, traditional approaches often generate unnatural results.

In shoot 'em up games, players control a character or a spacecraft, usually in vertical or horizontal directions, shooting enemies while dodging their attacks. These games are further subdivided into various types, such as vertical-scrolling shooters, horizontal-scrolling shooters, bullet hell shooters, and multidirectional shooters, based on how projectiles move across the screen. Few studies delve into employing PCGML algorithms for this game category, with even fewer applying GANs. The only research conducted by Wang et al. [60] explores projectile pattern generation in an indie game to validate the approach. The generation simulates the various motion patterns of the projectiles. The model is trained based on one instant of time to learn the representation of the projectile at the next instant.

4.5 Discussion

In summary, Table 1 shows the level generation problems, highlighting the work by game genre. We have identified and pointed out some of the issues in the literature on level generation explored using GAN techniques.

Table 1 Overview of literature associated problems in level generation research and categorized by game genre

Problem		Platform	Dungeon	Puzzle	Other
Level generation	Entire map	[47, 54, 65, 79]	[48, 49, 56, 59, 71–73, 80]	[63, 77, 78]	[46, 54, 58, 61, 62, 79, 81–83]
	Segments/ Rooms	[45, 53, 57, 66]	[56, 74, 76, 84–87]		
	Iterative/ Dynamic		[60, 76]		[60]
	Multiple games	[45]	[80, 86]		
	Realistic maps				[46, 85]
Structural cohesion	Game elements repair/ correct generation	[57, 65–67]	[56, 74, 88]	[63, 77, 78]	
	Connectivity between segments/rooms	[52, 57, 67]	[56, 76]		
	Connected path	[65]	[72, 73]		
Generation control	Elements adjustment	[68]	[68, 84]	[77]	
	Layout adjustment		[56]	[77, 78]	[60, 81–83]
	Difficulty adjustment	[47, 53, 67]			

We categorized these works under 3 main tasks: *Level generation*, *Structural cohesion* and *Generation control*. Under the *Level generation* category, 5 main problem areas have been identified, these are *Entire map*, *Segments/Rooms*, *Iterative/Dynamic*, *Multiple games* and *Realistic maps*. For *Structural cohesion* category, 3 main problem areas have been identified, such as *Game elements repair/correct generation*, *Connectivity between segments/rooms* and *Connected path*. And the *Generation control* category, we identified 3 main problem areas as *Elements adjustment*, *Layout adjustment* and *Difficulty adjustment*.

Level Generation addresses general approaches to different strategies for generate levels task. *Entire Map* refers to works focused on creating complete levels across various game genres. *Segments/Rooms* involves the generation of segments or individual rooms within levels. *Iterative/Dynamic* is related to iterative or dynamic generation methods that adjust the level over time. *Multiple Games* refers to approaches that have been generalized to different types of games. And *Realistic Maps* focuses on the realistic environments generations, using real data.

Works addressing the *Structural cohesion* of levels are categorized based on mechanisms for repairing or modifying GAN to enforce specific behavior in the generation.*Game elements repair/correct generation* refers to works that focus on the correct arrangement of game elements according to their structure or some specific game design rule. *Connectivity between segments/rooms* addresses methods ensuring coherent connections between different segments or rooms, whether within segmented rooms or across entire maps. Additionally, *Connected path* encompasses approaches that ensure continuous, accessible paths throughout the level, supporting seamless navigation across all areas.

Finally, *Generation control* category addresses works focusing on the controllability of level generation. *Elements Adjustment* includes methods for controlling the placement and arrangement of game elements within the level. *Layout Adjustment* refers to modifications of the terrain topology, ensuring that the layout aligns with specific design requirements. Lastly, *Difficulty Adjustment* refers to approaches that manage and adjust the difficulty level, ensuring an appropriate challenge for players.

4.5.1 Level generation

Generating a level may involve constructing the entire layout or generating segments, such as rooms or smaller-level sections. Ensuring continuity between segments or rooms and satisfying pathing criteria for player navigation across entire maps or terrain is a open problem. Given the nature of the algorithm and the volume of training data (see Section 5), developing a strategy to encourage learning for this type of task is quite complex. Works in the field address this issue through specialized models based on segment directions [57] or some condition [52], temporal models using known inputs [76], or by combining algorithms to maintain the cohesion of the level [45, 56, 66, 67]. Furthermore, some works apply evolutionary algorithms [65, 68, 71] to explore the GAN's latent space, adjusting the arrangement of elements to improve gameplay. These adjustments aim to correct structural flaws to ensure playable and accessible levels. Such issues are commonly encountered in platformers, dungeon crawlers, open-world games, and racing games.

In some generator models, the generation process has been adapted by treating time as a dependent variable. This approach is particularly relevant for tasks that require specific behaviors at different time instances, such as generating bullet patterns in shoot 'em up games or simulating game environments frame by frame [76]. Kim et al. [76] utilized 40,000 Pac-Man samples, each comprising 18 or 32 frames, for training their GAN. The work most similar in level generation using video data is that of Bruce et al. [89]. We believe that the

significant data requirements for training have constrained the number of studies exploring similar methodologies. In shoot 'em up games, Wang et al. [60] highlight the challenge of data availability. To address this, they implemented a bullet hell scenario to train their GAN. The limited research highlights a gap in the field, suggesting that further exploration and innovative solutions are needed to overcome the data challenges and advance the generation techniques for time-dependent game scenarios.

4.5.2 Structural cohesion

As mentioned earlier, a challenge with GANs lies in their capability to generate all game elements for maps, including structures, enemies, collectibles, etc. While GAN-based approaches for level generation often produce playable results, there is no assurance of completeness in their output. Generation becomes more challenging when the generated sample must adhere to specific physical constraints [63]. Based on the reviewed literature, integrating a repair technique [56, 67, 74] becomes necessary to make minimally playable adjustments to invalid levels and to ensure coherence in GAN-generated content.

Current literature has not yet fully resolved this problem, demonstrating alternative strategies that involve modifications to the GAN's loss function or architecture [66, 88], larger datasets [48, 63, 76, 84], or the application of evolutionary algorithms [65, 68, 71]) to search for representations that satisfy fitness functions, including specific structures or constraints of game elements see Section 5).

4.5.3 Controllable generation

Given the challenges associated with ensuring that levels are both playable and accessible, there is a growing need for better control mechanisms in GAN-based content generation. Conditioning the GAN to regulate the occurrence of specific elements [68, 77, 84], control the layout [56, 60, 77, 78, 81–83], or adjust the level difficulty [47, 53, 67] may require the involvement of a designer. Mixed-initiative approaches [67, 68, 70, 77], such as those employed by Schrum et al. [68], offer co-creative solution by enabling designers to control specific aspects of the layout searching the latent space of GANs.

5 Implementations of GANs in games

Level generation strictly depends on the style of the game and its gameplay, so the data is structured in a manner that prioritizes training the generative model effectively. As previously stated, a representative and diverse training dataset is needed. The majority of research focuses on level generation for games like Super Mario Bros. and The Legend of Zelda, largely due to the availability of a database in the Video Game Level Corpus[4] (VGLC) [75]) and the competition opportunities [55, 90, 91].

The literature reveals a scenario of using different approaches and adaptations of architectures, loss functions, and training processes originating from prior work in image processing and tailored for specific applications such as level generation. The choice and implementation of loss functions play a fundamental role in model performance and the quality of results. These implementations influence GAN models' training convergence and stability and have implications for adhering to strict design rules and achieving diversity in generated levels.

[4] https://github.com/TheVGLC/TheVGLC

In the following sections, we present an overview of the state-of-the-art in implementing training using GANs. We compile and discuss work at each stage of development, highlighting key advancements and methodologies.

5.1 Data representation

Defining the data representation our model uses dictates the training and inference procedures and how the training dataset should be acquired and processed. Many studies on level generation using GANs source their data from the VGLC, which offers three annotation formats for levels: tile, graph, and vector, as shown in Fig. 5. The tile format is commonly used for tile-based games, where the level layout, segments, or rooms are organized on a grid and represented as individual tiles. Each tile corresponds to a specific game element: terrain, obstacles, characters, or collectibles. This tiled data is translated into integer values and encoded within a tensor structure, with dedicated channels for each game element. Graph representation is employed when there's a need to describe the specific composition of a segment or room and how it connects to adjacent areas. Current GAN-based approaches typically do not implement this graph-based approach, leaving algorithms like Graph Grammar responsible for the connectivity between rooms and segments. Vector data is an image format based on Scalable Vector Graphics (SVG). Elements such as platforms and walls are defined as line segments and discrete objects and handled as graphical primitives, detailing their position within the game level.

In an initial approach to GAN training for level generation, Giacomello et al. [48] used flat images, each representing different information: wall, floor, height, and things maps. An additional feature vector, extracted from the existing levels available in the *idgames archive*, is used as input to the conditional network. Other studies employ an alternative representation to design image-level data. Unlike tile-based representation, which usually uses one-hot encoding and resulting in sparse matrices, some works have used tokens-based encoding methods. This approach yields a dense array, allowing to represent information that considers relationships between neighboring elements. Token-based representation can be understood as pixels from the level image, but instead of being one-hot encoded, each token corresponds to a specific element or component of the level, encoded through methods like block2vec. This allows the model to understand and manipulate the level's structure and content. In the TOAD-GAN architecture [79], tokens hold significance in preserving information during downsampling processes.

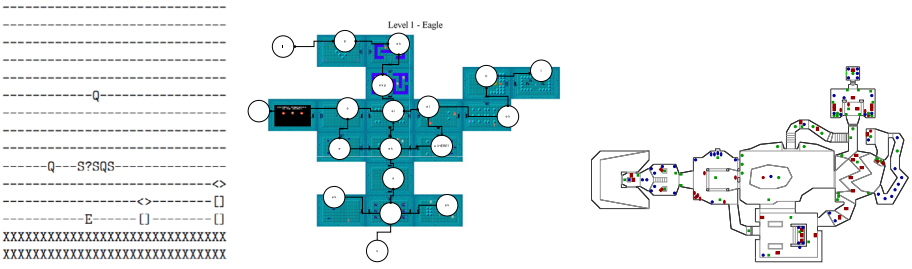

Fig. 5 The three data representations for levels, available in VGLC as: tile (left), graph (center), and vector (right) [75]

In procedural terrain generation, algorithms are designed to generate various natural features such as rivers, valleys, ridges, landscapes, and road networks. Certain studies have used real satellite images from OpenStreetMap [82], Shuttle Radar Topography Missions [85], and the United States Geological Survey [81] in a sketch-based approach. After careful preprocessing steps, specific features (e.g., trees, rivers, etc.) are extracted to enhance the training process and condition the results.

Another approach to using GANs is to generate parametric sequences as information to build levels, such as those used for bullet hell games [60]. This method allows learning the projectile movement dynamics common to this game genre. Like learning a sequence of data based on time, an alternative approach make use of videos as training data to learn the dynamic player movement in the Pac-Man game [76]. This method considers videos a sequence of frames organized across different time steps. Recently, Bruce et al. [89] presented a similar approach, using videos to learn the player movement dynamics for platform games.

5.2 Data augmentation

In data processing, it is well-known that deep learning algorithms are data-hungry, requiring a significant amount of data to learn patterns from the data distribution presented during training to enhance model quality. However, in the context of level generation applications, some studies highlight the ability to generate levels even when training with few samples: Schubert et al. [79] propose a one-shot learning approach, and Steckel et al. [65] analyzed how the quantity of data impacts the quality of the game Lode Runner. Their findings suggest that a few samples (e.g., 20 levels) are sufficient for training generative models while ensuring the generation of beatable platform levels. Limited studies delve into tuning or benchmarking networks based on the amount of data compared to the number of parameters in a network.

Even though Steckel et al. [65] emphasize that using limited data favors training, most studies draw from data processing experiences to enhance data expressiveness and improve generation quality. A practical way to obtain more data is through data augmentation techniques. In tile-based games, popular methods include transformations that maintain data integrity, such as 90° rotations (arbitrary angle rotations can readjust the levels to become invalid) and vertical/horizontal flips. Some approaches also incorporate morphological dilation for terrain data [72].

Feedback or recurrent feeding approaches are employed for data augmentation. For instance, Park et al. [59] use a GAN to train another GAN as a means of data augmentation. Meanwhile, Torrado et al. [84] adopt a bootstrapping approach to generate valid levels and enrich the training dataset corpus.

5.3 Architecture

In the literature, an extensive array of networks is observed in GAN training for level generation, with convolutional networks prevailing. Notably, research by Silva et al. [72] reveals the vulnerability of fully connected networks with linear layers, such as VanillaGAN, to mode collapse, underscoring the superiority of convolutional layers for level training. Similar findings can be found in the work of Usman et al. [87].

A seminal work employing GANs for level generation is Giacomello et al. [48] on PCG for DOOM, which employed WGAN-GP and conditional WGAN-GP architectures. These models exploited latent spaces and feature sets to generate image data representing various structures of DOOM levels. The study demonstrated an enhancement in the quality of gener-

ations with the conditional network, a trend corroborated by other findings [83], underscoring the efficacy of training with labels in guiding network learning and providing control over generated content.

Kumaran et al. [80] propose an architecture to generate game levels for multiple distinct games while sharing a common latent space. The architecture consists of a single generator, with the output of the final layer corresponding to the number of distinct games trained. Likewise, the number of discriminators corresponds individually to each game, serving to differentiate between them. Mirgati et al. [45] proposes an architecture that combines a Variational Autoencoder (VAE) network and a GAN to generate segments for multiple platform games. The VAE translates images into tile-based representations, while the GAN generates new examples from these representations.

TOAD-GAN [79] adapts the SinGAN architecture to function with token-based games, such as Super Mario Bros., by introducing modifications to the downsampling process and the determination of token importance, thereby enabling the generation of realistic 2D content. However, generating 3D content demands additional considerations due to the increased sample size and GPU space required. While TOAD-GAN focuses on generating content for 2D token-based games, World-GAN [58] extends this approach to 3D environments like Minecraft. World-GAN incorporates 3D convolutions and dense token embeddings, called block2vec, to manage 3D structures. Additionally, World-GAN evaluates generated content based on pattern similarity, variability, and the handling of rare tokens. In subsequent work, Awiszus et al. [92] extend World-GAN using BERT's natural language-based embeddings, concluding that this approach outperforms previous methods.

To generate levels for bullet hell games, Wang et al. [60] present two architectures for generating time series: TimeGAN and periodic spatial GAN. The TimeGAN architecture integrates a stochastic encoder and decoder, a temporal series adversarial network, and a regularization technique based on sequence prediction to ensure the temporal coherence of the generated series. Conversely, the periodic spatial GAN architecture aims to generate spatial data with periodic patterns, where periodicity is a prominent feature.

The works mentioned so far and their main characteristics, including a brief description of the architecture used (input and output formats), are summarized in Table 2.

5.4 Loss function

The selection of loss function in GAN training is crucial to the quality and diversity of generated content. Frequently employed loss functions such as Jensen-Shannon, Kullback-Leibler, and Wasserstein divergence have been extensively investigated. Through experimentation with DCGAN and WGAN models, WGAN demonstrates improved training stability and mitigation of mode-collapse issues [46, 72, 87]. Nonetheless, numerous studies continue to employ the DCGAN architecture due to its simplicity of implementation compared to WGAN. While WGAN has been shown to be more robust to hyperparameter changes, the parameter tuning process in DCGAN can be perceived as simpler because it does not require the more complex gradient penalty used in WGAN. However, further studies may provide additional insights into the ease of tuning parameters for both architectures.

Approaches applying GANs do not guarantee the complete accuracy of generated structures, requiring repair methodologies [56, 74]. In this context, Liello et al. [66] employ a CANs approach, where semantic loss penalizes the generator for producing invalid structures, thereby encouraging the generation of valid objects based on their structure. Similarly, Ferber et al. [88] employ a Neural Network to encode a constraint vector that is integrated

Table 2 Comprehensive overview of literature works detailing the work contribution, input and output data, and GAN architecture proposed

Generation	Conditional Input	Output	Arq.	Ref
Entire level	-	Tile-based	WGAN	[53, 65]
			DCGAN	[56, 59, 80]
			GAN, DCGAN, WGAN, WGANGP, DCGAN-SN, WGAN-SN, WGANGP-SN	[72]
			DCGAN-SN	[73]
			timeGAN + periodic-SpacialGAN	[60]
			WGAN-GP	[63]
			DCGAN + RL	[47]
			GenCO	[88]
			GlobalGAN	[78]
		Image-based	WGAN-GP + CMA-ES	[71]
			WGAN-GP	[48, 49]
		Block2vec	World-GAN	[58]
	Embedding feature vector	Tile-based	CESAGAN	[84]
	Mask	Tile-based	WGAN + CNN	[83]
	Token	Tile-based	TOAD-GAN	[54, 79]
	Embedding feature vector	Tile-based	WGAN-GP-PE	[77]
	Level features	Image-based	WGAN-GP	[48, 49]
Segment level	-	Tile-based	DCGAN	[86]
			MarioGAN + Fractional-Conv	[67]
			multiWGAN	[57]
			VAE-GAN	[45]
			WGAN	[68, 74]
		Image-based	GAN, DCGAN, WGAN	[87]
	Previous level	Tile-based		[52]
	Binary constraint vector	Tile-based	CAN	[66]
	Current image	Image-based	GameGAN	[76]
Terrain	Sketch	Tile-based	Pix2Pix	[81, 82]
	IMR	Heightmap	DCCWGAN	[85]
	-	Heightmap and texture	Pix2Pix	[61]
	-	Heightmap	DCGAN, WGAN, ProgGAN, VAE + WGAN	[46]
	Scatter maps	Image-based	GAN-terrain	[62]

into the loss function. Another approach involves training the network with valid and invalid data [73]. For each batch, the network is adjusted according to the distance between real and synthetic data, receiving a penalty for generating data that resemble invalid patterns.

5.5 Evaluation criteria

We conducted a comprehensive literature review on the metrics employed to assess the performance generation of the trained model. We identified several metrics that can be categorized into three main aspects: playability, variability, and topology. Playability metrics aim to ensure that generated levels are minimally playable and beatable. Variability metrics are intended to assess the diversity and novelty among the generated samples. Topology metrics evaluate the generation fidelity according to real data. This distinction between metrics allows for a more comprehensive and accurate analysis of the trained model's performance, as summarized in Table 3.

Following the traditional GAN approach in the image domain, Giacomello et al. [48, 49] represent game levels as image-based data rather than symbolic representations to generate entire levels. To evaluate the efficacy of this approach, they developed metrics to assess the topology of generated levels based on image properties. These metrics include visual similarity measures, such as Structural Similarity (SSIM) indices and pixel-wise comparisons, to quantify the resemblance between generated levels and ground truth images, using entropy, encoding error, and corner error as metrics for this evaluation. Voulgaris et al. [62] continue using scatter maps from satellite terrain images in the same domain. They validate the model through Normalized Histogram Intersection and an empirical evaluation, measuring the similarity of generations. However, approaches that use image-based data face challenges in assessing the quality of a level to validate its playability.

In some studies, it is not always clear which GVGAI[5] (General Video Game Artificial Intelligence) agents were employed to evaluate generations. Still, a predominance of search algorithms for pathfinding in platform-like and dungeon-like games is observed. Unlike Reinforcement Learning, the primary algorithms used in the literature are A* and Dijkstra, as they do not require agent training. These algorithms are sufficient for assessing the playability of a level, as they verify the player's accessibility to the challenges within the map. For instance, they check whether the avatar can reach the key and door [84], access the solution path [54], or evaluate connectivity criteria [57].

Specific heuristics can be tailored to access the correct generation of structures, the minimum/maximum quantity of game elements, or difficulty adjustment. In The Legend of Zelda, heuristics can be applied to count the number of avatars, keys, doors, and enemies in a given level or to evaluate whether all the edges of the level were generated [84]. Additionally, heuristics have been employed to assess the validity of the proportion of sampled objects in Super Mario Bros. [66] or to measure difficulty by the number of jumps required to complete the level [53].

Works have employed various strategies for evaluating the variability of level generators. For instance, in Angry Birds, a structured diversity heuristic is used, considering width, height, density, shape, and block frequency [63]. In The Legend of Zelda, Dijkstra's algorithm finds paths for each level, and the Manhattan distance is employed to assess similarity in rooms [80]. Additionally, Levenshtein Distance [80, 86] or minimal criteria novelty search [93] is used to assess novelty in level generations for The Legend of Zelda and ENGAGE [59] games. Di Liello et al. [66] use metrics to evaluate the novelty and uniqueness of sampled objects. Variance is also used to measure the variability of a set of samples [56, 73].

Some works are omitted from the Table due to their lack of evaluation of the output of their solvers [47, 61, 83] or because the metrics used do not directly assess the performance of a GAN model [67].

[5] https://github.com/GAIGResearch/GVGAI

Table 3 Overview of metrics for evaluating game level generation

	Playability		Variability		Topology	
Platform	A* Algorithm	[45, 52–54, 57, 65, 66, 79]	Tile Pattern KL-Divergence	[54, 79]	Accuracy	[45]
	Reinforcement Learning and Deep Q-networks	[47]	Novelty measure	[57]	Linearity	[45]
	Gameplay constraints	[53, 57, 66]	Level embedding	[54, 79]	Leniency	[45]
	Empirical evaluation	[68]	L1 norm	[66]	Interestingness	[45]
			Empirical evaluation	[68]		
Dungeon	A* Algorithm	[56, 74, 74, 84, 84]	Duplication measure	[74, 84]	Corner Error	[48]
	Agent from GVGAI	[80, 86]	Novelty search	[93]	Encoding Error	[48]
	Reinforcement Learning	[76]	Dijkstra's algorithm	[80]	Entropy	[48]
	Dijkstra's algorithm	[59]	Pixelwise variance	[56, 73]	Kolmogorov-Smirnov test [49, 71]	
	Gameplay constraints	[59, 72–74, 84, 87]	Diversity	[88]	L2 norm [71]	
	Long-term pixel consistency	[76]	Distances measures	[74, 80, 84]	Percentiles (generation controllability)	[49]
	Correspondence	[62]	Empirical evaluation	[68]	Structural Similarity (SSIM)	[48]
	Plausibility	[62]			Fidelity	[88]
	Empirical evaluation	[68]			Empirical evaluation	[46, 81, 82, 85]
Puzzle	Broken pieces	[77]	Width, height	[63]	Symmetry	[77, 78]
	Color-islands	[77]	Density, shape	[63]		
	Pieces distribution	[77]	Block frequency	[63]		
	Block destruction and block velocity	[63]				

Table 3 continued

	Playability		Variability		Topology	
Other	A* Algorithm	[54, 79]	Level embedding	[54, 79]	Fidelity	[88]
	Coverage	[60]	Tile Pattern KL-divergence	[54, 58, 79]		
	Mean momentum	[60]	Levenshtein distance	[58]		
	Shooting frequency	[60]	Diversity	[88]		

In summary, the choice and implementation of metrics for evaluating level generations, such as playability and variability, depend on the game's specific criteria and design goals. For instance, a dungeon game might prioritize metrics that evaluate the complexity of dungeon layouts, the balance of combat encounters, and the exploration of hidden secrets. Similarly, the definition of diversity within a game can be interpreted differently depending on the design intentions, encompassing aspects such as dungeon environments, enemy types, and loot variety.

5.6 User studies and feedback

Few studies have conducted user experiments to evaluate the quality of generators. However, those who did not conduct experimentation emphasize its significance in evaluating quality [45, 47, 54]. Generally, approaches proposing generator enhancements underscore the increase in the percentage of valid samples as a measure of quality, employing playability metrics described in the previous section.

Gutierrez et al. [56] conducted a user study to evaluate the level generation potential of GAN with and without Graph Grammar. The study assessed each approach's enjoyability, complexity, novelty, organization, and challenges. Quantitative and qualitative data indicated no significant differences between dungeon types regarding enjoyment, challenge, or complexity, except for room organization, where rooms generated by GAN without Graph Grammar were notably less organized.

In another work, Schrum et al. [68] conducted a study to evaluate interactive evolution and manual exploration of latent space, separately and combined, to generate new levels. The authors found that users appreciated both aspects and slightly preferred direct exploration. However, combining these features led to the discovery of even better levels. User feedback also suggested the system's potential as a commercial design tool with further enhancements.

In terrain generation, Ramos et al. [85] conducted a study with 79 participants, concluding that participants could not distinguish the ground-based and height map images generated by their approach from real data. Guerin et al. [81] conducted a study with 5 participants to evaluate whether the generator worked as expected. The results demonstrated a certain acceptance of the proposed generator.

5.7 Discussion

We can extract several vital points from the literature reviewed in the survey regarding the challenges and advancements in the area of GANs for level generation.

5.7.1 Research databases

VGLC and GVGAI are the main game databases available in literature, but with few samples of each game. It is widely accepted that the availability of data is essential for improving GAN training in image processing area. However, Steckel et al. [65] suggest that a large amount of data is not essential for training GANs. However, there are few works that explore some data augmentation strategies. More studies are needed to determine the optimal amount of data required for training, the impact of data augmentation techniques on the final results, and how to adapt these techniques to different game genres.

In the data pre-processing step, we find some data representations (e.g. tiles, graphs, vectors, images, videos) present in the literature for training GANs. We observed that most

games use a tile representation to describe the level map, whether it is the entire map or just a segment of the level. In non-linear games, the level can appear in the graph representation, to better describe specific objectives of each level segment, which favors the general organization of the level. In racing, city-based or open-world games, satellite images are commonly used to enhance realism from the generated maps. There are few works that address these game genres, but we can find related works in the area of terrain generation, highlighting the intersection of GANs with other fields such as geospatial analysis. Such data are available in OpenStreetMap [82], Shuttle Radar Topography Missions [85], and the United States Geological Survey [81].

Future research should combine these game-specific representations with other image-modalities, such as sketch, mask, segmentation, to enhance the validity and diversity of generated levels. Additionally, a more in-depth exploration of hybrid models that integrate both tile-based and non-tile-based representations could lead to more robust GAN architectures.

5.7.2 Training data

Unbalanced data is expected to be encountered in level design, where certain structures only occupy a small fraction of the sample and have an inherent relationship with another distant structure. This problem has been addressed by few works. Torrado et al. [84] proposed adjusting the network architecture using a self-attention mechanism to give importance to structures with lower occurrence. In contrast, Awiszus et al. [58, 92] modified sample rates through an equation based on occurrence, oversampling the low probabilities. Volz et al. [78] highlights that learning both global and local features of data is a significant challenge for GANs. This difficulty is due to the inherent limitations of convolutional networks. While this observation is consistent with current understanding, further research is necessary to validate this statement across various games.

The literature includes examples of methods designed to capture these features, such as self-attention [84] mechanisms, multi-scale GANs [54, 58, 79] and implementation of U-Net [61, 81, 82]. Addressing this challenge is crucial further investigation to better understand how GANs learn and which features most effectively guide their learning process.

Future directions should focus on investigating more sampling strategies that can efficiently handle unbalanced data without sacrificing the representation of rare game elements or structures. Furthermore, exploring hybrid architectures, such as combining adversarial training with additional deep learning approaches or other techniques, could yield more robust models capable of better capturing both global and local dependencies in the data. Additionally, the development of multi-modal architectures that leverage diverse data representations could enhance the generalization capabilities across different types of datasets.

5.7.3 Architecture

Generating data obeying specific constraints of the dataset is not an easy task. Heuristic methods and repair mechanisms are often needed to ensure the validity of generated levels, addressing issues such as the generation of invalid structures, or unbeatable levels. Combining GANs with other techniques, such as evolutionary algorithms, reinforcement learning, and solvers, can help improve the generation process and address limitations. Various architectural modifications, such as convolutional networks and specific loss functions like Wasserstein divergence, have been proposed to enhance the performance and stability of GANs in level generation.

The literature review identified studies comparing some GAN implementations [46, 72, 87], highlighting that WGAN and/or its variant with gradient penalty (WGAN-GP) tend to produce superior generation outcomes. Convolutional networks utilizing the Wasserstein loss function are predominantly employed in the literature, although traditional DCGANs continue to be used in some cases. Architectural modifications encompass adjustments to regularization techniques, modifications in convolutional kernels, and the replacement of traditional networks with more advanced architectures, such as multi-scale GANs [54, 58, 79] and pix2pix models enhanced with U-Net structures [61, 81, 82]. Further refinements include changes to the loss function, incorporating approaches like semantic loss [66] or the integration of penalty-based methods [73] to enhance training stability and generation quality.

Although the survey highlights a significant number of related researches, there is no clear consensus on the optimal architecture choice. The diversity of games and their unique challenges significantly influence problem modeling, making the selection of the most suitable architecture highly context-dependent.

Future work could focus on developing hybrid architectures that integrate the strengths of multiple models or employ multiple discriminators to better address diverse level design requirements. This could include exploring hybrid or multi-modal models, as mentioned on Section 5.7.2, to leverage varied data representations and enhance the validity and quality of generated levels.

5.7.4 Multiple games

The difficulty of training a single GAN model to generate levels for multiple distinct games is highlighted. Approaches like modifying the generator output layer to accommodate different games have been explored. Different game genres pose unique challenges for GAN-based level generation. For instance, platform games require maintaining cohesion in-game elements, while dungeon games need to ensure connectivity between segments or rooms. Similarly, research efforts are dedicated to addressing the issue of blending levels [94–97], although limited studies employ GANs for this purpose.

Future studies could focus on developing domain-specific layers or modular architectures within GANs that could be fine-tuned for different genres. For instance, the integration of game mechanics-specific knowledge into the generation process, such as platformer physics or dungeon connectivity rules.

5.7.5 Evaluation metrics

The evaluation of GAN-generated levels is generally quite nuanced. Playability, variability, and topology metrics are the most commonly used. Playability metrics focus on ensuring that levels are functional and beatable, often using algorithms like A* and Dijkstra to verify pathfinding feasibility. Variability measures aim to capture the diversity and uniqueness of the generated levels, employing metrics such as novelty measures, level embedding, and pixelwise variance. Topology metrics assess the generation fidelity by comparing generated levels to real data through visual similarity measures like Structural Similarity (SSIM) indices, entropy, and encoding error. The literature indicates that no single architecture consistently excels across all evaluation criteria. The choice often depends on specific game requirements and design goals.

There are trends in the use of metrics for some game genres, but evaluating quality goes beyond aesthetic comparison. User studies, though limited, are emphasized as significant

for validating the effectiveness of level generators. User feedback can provide insights into generated levels' enjoyability, complexity, novelty, and organization. Some studies indicate the potential for GAN-based systems to be used as commercial design tools, with further enhancements based on user feedback. Investigating user experience is necessary, especially when levels are dynamically generated.

Overall, the field still faces challenges in standardizing evaluation criteria across diverse game genres and design requirements. Further research is needed to validate these metrics more broadly, and further investigation into the impact of GAN-generated content on user engagement and satisfaction could provide valuable insights for improving level generation models. Additionally, developing real-time user feedback systems could also be an important avenue for refining level generation processes.

6 Conclusion

This survey has explored the application of GANs in procedural content generation for digital game levels, dividing the work into two main perspectives: game design and algorithmic approaches. We identified completed studies in each section, areas requiring enhancement, emerging trends, and unexplored directions.

Our findings demonstrate that GANs hold significant potential for automatic level generation by learning complex patterns from game levels. GANs have been effectively used across various game genres due to their ease of implementation. However, challenges remain in generating playable and diverse levels. We also identified a study where GANs are not well-suited, such as in generating levels with symmetrical patterns.

Key takeaways from this survey include the identification of gaps requiring further investigation, such as the need for more extensive user studies to validate the quality of the proposed approaches effectively. Additionally, underexplored issues in GAN training, such as the optimal quantity of data and the impact of data augmentation techniques, require further study. The difficulty of training a single generalist model to generate different games and the need for better evaluation metrics are also significant challenges.

To address these challenges, future research could benefit from employing conditional GANs, which allow more controlled generation by conditioning on specific attributes or level features. This approach may help refine the generation process to produce levels that better meet game-specific requirements. Additionally, exploring multi-modal GANs could further enhance diversity in content generation, enabling the creation of multi-faceted game environments. Combining GANs with evolutionary algorithms or other methods to correct invalid samples could enhance the generation of more playable and varied levels.

In conclusion, while challenges must be addressed, GANs offer a powerful tool for procedural content generation in games. Their potential for creating diverse, engaging, and playable game levels remains significant, and further exploration and validation through user studies will be crucial for harnessing their full capabilities.

Data Availability Statement The data supporting the findings of this study are available in the articles cited in the References section. Each analyzed article can be accessed through the following digital libraries: *ACM Digital Library* and *IEEE Xplore*, or the search tool *Google Scholar*. The images presented are properly referenced, and the data can be accessed in the respective works or in the public repository https://github.com/TheVGLC/TheVGLC.

Declarations

Competing Interests This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001 and Fundação de Amparo a Pesquisa do Estado do Rio Grande do Sul (FAPERGS) through grant 21/2551-0002087-3.

References

1. Amato A (2017) In: Korn O, Lee N (eds) Procedural content generation in the game industry. Springer, Cham, pp 15–25. https://doi.org/10.1007/978-3-319-53088-8_2
2. Font JM, Izquierdo R, Manrique D, Togelius J (2016) Constrained level generation through grammar-based evolutionary algorithms. In: Squillero G, Burelli P (eds) Applications of evolutionary computation. Springer, Cham, pp 558–573
3. Valls-Vargas J, Zhu J, Ontañón S (2017) Graph grammar-based controllable generation of puzzles for a learning game about parallel programming. In: Proceedings of the 12th international conference on the foundations of digital games. FDG '17. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3102071.3102079
4. Linden R, Lopes R, Bidarra R (2021) Designing procedurally generated levels. Proc AAAI Conf Artif Intell Interactive Digital Entertainment 9(3):41–47. https://doi.org/10.1609/aiide.v9i3.12592
5. Togelius J, Yannakakis GN, Stanley KO, Browne C (2011) Search-based procedural content generation: a taxonomy and survey. IEEE Trans Comput Intell AI Games 3(3):172–186
6. Khalifa A, Togelius J, Green MC (2022) Mutation models: learning to generate levels by imitating evolution. In: Proceedings of the 17th international conference on the foundations of digital games. FDG '22. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3555858.3563267
7. Bagus Harisa A, Tai W-K (2022) Pacing-based procedural dungeon level generation: alternating level creation to meet designer's expectations. Int J Comput Digit Syst 12(1):401–416
8. Hojatoleslami MR, Zamanifar K, Zojaji Z (2024) Gfgda: general framework for generating dungeons with atmosphere. Multimed Tools Appl, 1–35. https://doi.org/10.1007/s11042-024-18833-5
9. Johnson L, Yannakakis GN, Togelius J (2010) Cellular automata for real-time generation of infinite cave levels. In: Proceedings of the 2010 workshop on procedural content generation in games. PCGames '10. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/1814256.1814266
10. Adams C, Louis S (2017) Procedural maze level generation with evolutionary cellular automata. In: 2017 IEEE Symposium Series on Computational Intelligence (SSCI), pp 1–8. https://doi.org/10.1109/SSCI.2017.8285213
11. Smith AM, Mateas M (2011) Answer set programming for procedural content generation: a design space approach. IEEE Trans Comput Intell AI Games 3(3):187–200. https://doi.org/10.1109/TCIAIG.2011.2158545
12. Lee V, Partlan N, Cooper S (2020) Precomputing player movement in platformers for level generation with reachability constraints. In: AIIDE workshops
13. Guzdial M, Snodgrass S, Summerville AJ (2022) Constraint-Based PCGML approaches. Springer, Cham, pp 51–66. https://doi.org/10.1007/978-3-031-16719-5_5
14. Mortazavi F, Moradi H, Vahabie A-H (2024) Dynamic difficulty adjustment approaches in video games: a systematic literature review. Multimed Tools Appl, 1–48. https://doi.org/10.1007/s11042-024-18768-x
15. Isola P, Zhu J-Y, Zhou T, Efros AA (2017) Image-to-image translation with conditional adversarial networks. CVPR
16. Zhu J-Y, Park T, Isola P, Efros AA (2017) Unpaired image-to-image translation using cycle-consistent adversarial networks. In: Computer Vision (ICCV), 2017 IEEE international conference on
17. Brock A, Donahue J, Simonyan K (2019) Large scale GAN training for high fidelity natural image synthesis
18. Karras T, Laine S, Aila T (2021) A style-based generator architecture for generative adversarial networks. IEEE Trans Pattern Anal Mach Intell 43(12):4217–4228. https://doi.org/10.1109/TPAMI.2020.2970919
19. Kodali N, Abernethy J, Hays J, Kira Z (2017) On convergence and stability of gans. arXiv preprint arXiv:1705.07215
20. Thanh-Tung H, Tran T, Venkatesh S (2019) Improving generalization and stability of generative adversarial networks. arXiv preprint arXiv:1902.03984

21. Salimans T, Goodfellow IJ, Zaremba W, Cheung V, Radford A, Chen X (2016) Improved techniques for training gans. CoRR. abs/1606.03498. arXiv:1606.03498
22. Zakaria Y, Fayek M, Hadhoud M (2023) Procedural level generation for sokoban via deep learning: an experimental study. IEEE Trans Games 15(1):108–120. https://doi.org/10.1109/TG.2022.3175795
23. Bond-Taylor S, Leach A, Long Y, Willcocks CG (2022) Deep generative modelling: a comparative review of vaes, gans, normalizing flows, energy-based and autoregressive models. IEEE Trans Pattern Anal Mach Intell 44(11):7327–7347. https://doi.org/10.1109/TPAMI.2021.3116668
24. Hendrikx M, Meijer S, Van Der Velden J, Iosup A (2013) Procedural content generation for games: a survey. ACM Trans Multimed Comput Commun Appl (TOMM) 9(1):1–22
25. Smelik RM, Tutenel T, Bidarra R, Benes B (2014) A survey on procedural modelling for virtual worlds. Comput Graph Forum 33(6):31–50. https://doi.org/10.1111/cgf.12276. https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.12276
26. Shaker N, Togelius J, Nelson MJ (2016) Procedural content generation in games: a textbook and an overview of current research. Springer, Switzerland. https://doi.org/10.1007/978-3-319-42716-4
27. Summerville A, Snodgrass S, Guzdial M, Holmgård C, Hoover AK, Isaksen A, Nealen A, Togelius J (2018) Procedural content generation via machine learning (pcgml). IEEE Trans Games 10(3):257–270
28. Liu J, Snodgrass S, Khalifa A, Risi S, Yannakakis GN, Togelius J (2021) Deep learning for procedural content generation. Neural Comput Appl 33(1):19–37
29. Linden R, Lopes R, Bidarra R (2014) Procedural generation of dungeons. IEEE Trans Comput Intell AI Games 6(1):78–89. https://doi.org/10.1109/TCIAIG.2013.2290371
30. Viana BM, Santos SR (2021) Procedural dungeon generation: a survey. J Interact Syst 12(1):83–101
31. De Kegel B, Haahr M (2020) Procedural puzzle generation: a survey. IEEE Trans Games 12(1):21–40. https://doi.org/10.1109/TG.2019.2917792
32. Sarkar A, Guzdial M, Snodgrass S, Summerville A, Machado T, Smith G (2023) Procedural content generation via knowledge transformation (pcg-kt). IEEE Trans Games
33. Khalifa A, Bontrager P, Earle S, Togelius J (2020) Pcgrl: procedural content generation via reinforcement learning. Proc AAAI Conf Artif Intell Interactive Digital Entertainment 16(1):95–101. https://doi.org/10.1609/aiide.v16i1.7416
34. Gisslén L, Eakins A, Gordillo C, Bergdahl J, Tollmar K (2021) Adversarial reinforcement learning for procedural content generation. In: 2021 IEEE Conference on Games (CoG), pp 1–8. https://doi.org/10.1109/CoG52621.2021.9619053
35. Bontrager P, Togelius J (2021) Learning to generate levels from nothing. In: 2021 IEEE Conference on Games (CoG). IEEE Press, ???, pp 1–8. https://doi.org/10.1109/CoG52621.2021.9619131
36. Hughes RT, Zhu L, Bednarz T (2021) Generative adversarial networks-enabled human-artificial intelligence collaborative applications for creative and design industries: a systematic review of current approaches and trends. Front Artif Intell 4. https://doi.org/10.3389/frai.2021.604234
37. Goodfellow IJ, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A, Bengio Y (2014) Generative adversarial nets. In: Proceedings of the 27th international conference on neural information processing systems - Volume 2. NIPS'14. MIT Press, Cambridge, MA, USA, pp 2672–2680. https://doi.org/10.48550/arXiv.1406.2661
38. Radford A, Metz L, Chintala S (2015) Unsupervised representation learning with deep convolutional generative adversarial networks. arXiv preprint arXiv:1511.06434
39. Gulrajani I, Ahmed F, Arjovsky M, Dumoulin V, Courville A (2017) Improved training of Wasserstein GANs. arXiv. https://doi.org/10.48550/ARXIV.1704.00028. https://arxiv.org/abs/1704.00028
40. Miyato T, Kataoka T, Koyama M, Yoshida Y (2018) Spectral normalization for generative adversarial networks. arXiv preprint arXiv:1802.05957. https://doi.org/10.48550/arXiv.1802.05957
41. Strelcenia E, Prakoonwit S (2023) A survey on gan techniques for data augmentation to address the imbalanced data issues in credit card fraud detection. Mach Learn Knowl Extr 5(1):304–329
42. Sampath V, Maurtua I, Aguilar Martin JJ, Gutierrez A (2021) A survey on generative adversarial networks for imbalance problems in computer vision tasks. J Big Data 8:1–59
43. Xia W, Zhang Y, Yang Y, Xue J-H, Zhou B, Yang M-H (2022) Gan inversion: a survey. IEEE Trans Pattern Anal Mach Intell 45(3):3121–3138
44. Borji A (2022) Pros and cons of gan evaluation measures: new developments. Comput Vis Image Underst 215:103329
45. Mirgati N, Guzdial M (2023) Joint level generation and translation using gameplay videos. In: 2023 IEEE Conference on Games (CoG), pp 1–10. https://doi.org/10.1109/CoG57401.2023.10333193
46. Nunes V, Dias J, Santos PA (2023) Gan-based content generation of maps for strategy games. arXiv preprint arXiv:2301.02874

47. Rajabi M, Ashtiani M, Minaei Bidgoli B, Davoodi O (2021) A dynamic balanced level generator for video games based on deep convolutional generative adversarial networks. Scientia Iranica 28(Special issue on collective behavior of nonlinear dynamical networks):1497–1514
48. Giacomello E, Lanzi PL, Loiacono D (2018) Doom level generation using generative adversarial networks. In: 2018 IEEE Games, Entertainment, Media Conference (GEM). IEEE, pp 316–323
49. Giacomello E, Lanzi PL, Loiacono D (2023) An analysis of doom level generation using generative adversarial networks. Entertain Comput 46:100549
50. Schrum J, Volz V, Risi S (2020) Cppn2gan: combining compositional pattern producing networks and gans for large-scale pattern generation. In: Proceedings of the 2020 genetic and evolutionary computation conference. GECCO '20. Association for Computing Machinery, New York, NY, USA, pp 139–147. https://doi.org/10.1145/3377930.3389822
51. Fontaine MC, Liu R, Khalifa A, Modi J, Togelius J, Hoover AK, Nikolaidis S (2021) Illuminating mario scenes in the latent space of a generative adversarial network. Proc AAAI Conf Artif Intell 35(7):5922–5930. https://doi.org/10.1609/aaai.v35i7.16740
52. Nam S, Hsueh C-H, Ikeda K (2023) Procedural content generation of super mario levels considering natural connection. In: 2023 20th International Joint Conference on Computer Science and Software Engineering (JCSSE). IEEE, pp 291–296
53. Volz V, Schrum J, Liu J, Lucas SM, Smith A, Risi S (2018) Evolving mario levels in the latent space of a deep convolutional generative adversarial network. In: Proceedings of the genetic and evolutionary computation conference, pp 221–228
54. Awiszus M, Schubert F, Rosenhahn B (2020) Toad-gan: coherent style level generation from a single example. In: Proceedings of the Sixteenth AAAI conference on artificial intelligence and interactive digital entertainment. AIIDE'20. AAAI Press, ???. https://doi.org/10.48550/arXiv.2008.01531
55. Shaker N, Togelius J, Yannakakis GN, Weber B, Shimizu T, Hashiyama T, Sorenson N, Pasquier P, Mawhorter P, Takahashi G et al (2011) The 2010 mario ai championship: level generation track. IEEE Trans Comput Intell AI Games 3(4):332–347
56. Gutierrez J, Schrum J (2020) Generative adversarial network rooms in generative graph grammar dungeons for the legend of zelda. In: 2020 IEEE Congress on Evolutionary Computation (CEC). IEEE, pp 1–8
57. Capps B, Schrum J (2021) Using multiple generative adversarial networks to build better-connected levels for mega man. In: Proceedings of the genetic and evolutionary computation conference. GECCO '21. Association for Computing Machinery, New York, NY, USA, pp 66–74. https://doi.org/10.1145/3449639.3459323
58. Awiszus M, Schubert F, Rosenhahn B (2021) World-gan: a generative model for minecraft worlds. In: 2021 IEEE Conference on Games (CoG). IEEE, pp 1–8
59. Park K, Mott BW, Min W, Boyer KE, Wiebe EN, Lester JC (2019) Generating educational game levels with multistep deep convolutional generative adversarial networks. In: 2019 IEEE Conference on Games (CoG). IEEE, pp 1–8
60. Wang Z, Liu J, Yannakakis GN (2021) Keiki: towards realistic danmaku generation via sequential gans. In: 2021 IEEE Conference on Games (CoG). IEEE, pp 01–04
61. Beckham C, Pal C (2017) A step towards procedural terrain generation with GANs
62. Voulgaris G, Mademlis I, Pitas I (2021) Procedural terrain generation using generative adversarial networks. In: 2021 29th European Signal Processing Conference (EUSIPCO). IEEE, pp 686–690
63. Abraham F, Stephenson M (2023) Utilizing generative adversarial networks for stable structure generation in angry birds. In: Proceedings of the AAAI conference on artificial intelligence and interactive digital entertainment, vol 19, pp 2–12
64. Merino T, Charity M, Togelius J (2023) Interactive latent variable evolution for the generation of minecraft structures. In: Proceedings of the 18th international conference on the foundations of digital games. FDG '23. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3582437.3587208
65. Steckel K, Schrum J (2021) Illuminating the space of beatable lode runner levels produced by various generative adversarial networks. In: Proceedings of the genetic and evolutionary computation conference companion, pp 111–112
66. Di Liello L, Ardino P, Gobbi J, Morettin P, Teso S, Passerini A (2020) Efficient generation of structured objects with constrained adversarial networks. Adv Neural Inf Process Syst 33:14663–14674
67. Wang Z, Liu J (2022) Online game level generation from music. In: 2022 IEEE Conference on Games (CoG). IEEE, pp 119–126
68. Schrum J, Gutierrez J, Volz V, Liu J, Lucas S, Risi S (2020) Interactive evolution and exploration within latent level-design space of generative adversarial networks. In: Proceedings of the 2020 genetic and evolutionary computation conference, pp 148–156

69. Schrum J, Capps B, Steckel K, Volz V, Risi S (2022) Hybrid encoding for generating large scale game level patterns with local variations. IEEE Trans Games 15(1):46–55
70. Morettin P, Passerini A, Teso S et al (2021) Co-creating platformer levels with constrained adversarial networks. In: CEUR workshop proceedings, vol 2903. CEUR workshop proceedings
71. Giacomello E, Lanzi PL, Loiacono D (2019) Searching the latent space of a generative adversarial network to generate doom levels. In: 2019 IEEE Conference on Games (CoG). IEEE, pp 1–8
72. Silva DF, Torchelsen RP, Aguiar MS et al (2023) Dungeon level generation using generative adversarial network: an experimental study for top-down view games. In: Anais do L Seminário Integrado de Software e Hardware. SBC, pp 95–106
73. Silva DF, Torchelsen RP, De Aguiar MS (2023) How to improve the quality of gan-based map generators. In: Proceedings of the 22nd brazilian symposium on games and digital entertainment, pp 106–113
74. Zhang H, Fontaine MC, Hoover AK, Togelius J, Dilkina B, Nikolaidis S (2020) Video game level repair via mixed integer linear programming. In: Proceedings of the sixteenth AAAI conference on artificial intelligence and interactive digital entertainment. AIIDE'20. AAAI Press, ???. https://doi.org/10.48550/arXiv.2010.06627
75. Summerville AJ, Snodgrass S, Mateas M, Ontanón S (2016) The vglc: the video game level corpus. arXiv preprint arXiv:1606.07487
76. Kim SW, Zhou Y, Philion J, Torralba A, Fidler S (2020) Learning to simulate dynamic environments with gamegan. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp 1231–1240
77. Hald A, Hansen JS, Kristensen J, Burelli P (2020) Procedural content generation of puzzle games using conditional generative adversarial networks. In: Proceedings of the 15th international conference on the foundations of digital games, pp 1–9
78. Volz V, Justesen N, Snodgrass S, Asadi S, Purmonen S, Holmgård C, Togelius J, Risi S (2020) Capturing local and global patterns in procedural content generation via machine learning. In: 2020 IEEE Conference on Games (CoG), pp 399–406. https://doi.org/10.1109/CoG47356.2020.9231944
79. Schubert F, Awiszus M, Rosenhahn B (2021) Toad-gan: a flexible framework for few-shot level generation in token-based games. IEEE Trans Games 14(2):284–293
80. Kumaran V, Mott B, Lester J (2019) Generating game levels for multiple distinct games with a common latent space. In: Proceedings of the AAAI conference on artificial intelligence and interactive digital entertainment, vol 15, pp 102–108
81. Guérin É, Digne J, Galin E, Peytavie A, Wolf C, Benes B, Martinez B (2017) Interactive example-based terrain authoring with conditional generative adversarial networks. ACM Trans Graph 36(6):228–1
82. Kelvin LZ, Bhojan A (2020) Procedural generation of roads with conditional generative adversarial networks. In: ACM SIGGRAPH 2020 Posters, pp 1–2
83. Ping K, Dingli L (2020) Conditional convolutional generative adversarial networks based interactive procedural game map generation. In: Advances in information and communication: proceedings of the 2020 Future of Information and Communication Conference (FICC), vol 1. Springer, pp 400–419
84. Rodriguez Torrado R, Khalifa A, Cerny Green M, Justesen N, Risi S, Togelius J (2020) Bootstrapping conditional gans for video game level generation. In: 2020 IEEE Conference on Games (CoG), pp 41–48. https://doi.org/10.1109/CoG47356.2020.9231576
85. Ramos N, Santos P, Dias J (2023) Dual critic conditional wasserstein gan for height-map generation. In: Proceedings of the 18th international conference on the foundations of digital games, pp 1–4
86. Irfan A, Zafar A, Hassan S (2019) Evolving levels for general games using deep convolutional generative adversarial networks. In: 2019 11th computer science and electronic engineering (CEEC). IEEE, pp 96–101
87. Usman F, Anwar SM, Rauf U (2023) Visual recognition for zelda content generation via generative adversarial network. In: 2023 3rd International Conference on Artificial Intelligence (ICAI). IEEE, pp 76–81
88. Ferber A, Zharmagambetov A, Huang T, Dilkina B, Tian Y (2023) Genco: generating diverse solutions to design problems with combinatorial nature. arXiv preprint arXiv:2310.02442
89. Bruce J, Dennis M, Edwards A, Parker-Holder J, Shi Y, Hughes E, Lai M, Mavalankar A, Steigerwald R, Apps C et al (2024) Genie: generative interactive environments. arXiv preprint arXiv:2402.15391
90. Perez-Liebana D, Samothrakis S, Togelius J, Schaul T, Lucas SM, Couëtoux A, Lee J, Lim C-U, Thompson T (2015) The 2014 general video game playing competition. IEEE Trans Comput Intell AI Games 8(3):229–243
91. Renz J, Ge X, Stephenson M, Zhang P (2019) Ai meets angry birds. Nat Mach Intell 1(7):328–328
92. Awiszus M, Schubert F, Rosenhahn B (2023) Wor(l)d-gan: toward natural-language-based pcg in minecraft. IEEE Trans Games 15(2):182–192. https://doi.org/10.1109/TG.2022.3153206

93. Lehman J, Stanley KO (2010) Revising the evolutionary computation abstraction: minimal criteria novelty search. In: Proceedings of the 12th annual conference on genetic and evolutionary computation, pp 103–110
94. Sarkar A, Yang Z, Cooper S (2019) Controllable level blending between games using variational autoencoders. In: Proceedings of the EXAG workshop at AIIDE
95. Sarkar A, Cooper S (2020) Sequential segment-based level generation and blending using variational autoencoders. In: Proceedings of the 15th international conference on the foundations of digital games. FDG '20. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3402942.3409604
96. Sarkar A, Cooper S (2021) Generating and blending game levels via quality-diversity in the latent space of a variational autoencoder. In: Proceedings of the 16th international conference on the foundations of digital games. FDG '21. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3472538.3472545
97. Sarkar A, Cooper S (2021) Dungeon and platformer level blending and generation using conditional vaes. In: Proceedings of the IEEE Conference on Games (CoG)